Multimodal Pareto-Optimal Path Routing

Course Project

SL203: Introduction to Algorithms and Software Programming

Shreya Ghosh, Soham Chakraborty, Rahul Dev Sharma

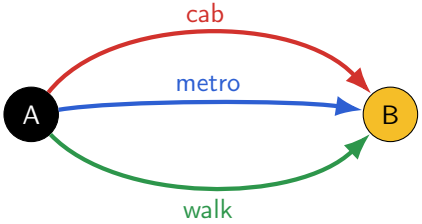
Instructor: Dr. Nagarajan Natarajan

April 30, 2026

1. Motivation: Routing Is a Choice Problem

A real trip from A to B

- Cab: fast and direct, but expensive.
- Metro: fast over distance, but needs walking.
- Walking: free, but slow and tiring.



Why should we care?

A route is not just geometry. It reflects time, money, effort, and convenience.

Decision dilemma

Fast, cheap, low-walk, and low-transfer rarely happen together.

2. Classical Shortest Path

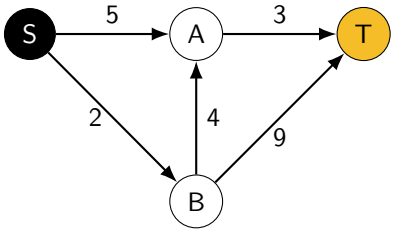
Intuition: Dijkstra

- Keep the best known distance to each node.
- Expand the node with smallest current distance.
- Stop when the destination is finalized.

Small mathematical formulation

For scalar edge weight w_e:

$$p^* = \arg\min_{p \in \mathcal{P}_{s,t}} \sum_{e \in p} w_e$$



Limitation

Dijkstra assumes one objective represents path quality.

3. Multi-Objective Problem

Conflict example

Path	Time	Cost
A	20 min	Rs. 200
B	50 min	Rs. 20

Which path is better?

The answer depends on context. That means scalar "best" is not always meaningful.

Vector cost

Instead of one number, assign each path a vector:

$$C(p) = (C_1(p), C_2(p), \ldots, C_k(p))$$

Why care?

We can preserve alternatives that are better for different users or situations.

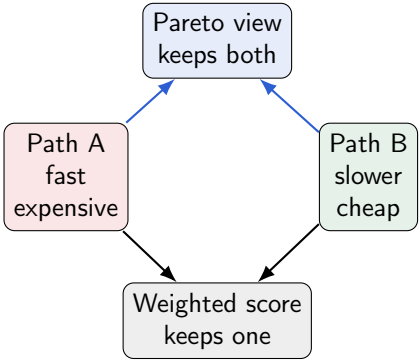
Why Dijkstra Fails Under Trade-offs

Same trip, two valid answers

Path	Time	Cost
A	20 min	Rs. 200
B	30 min	Rs. 20

Scalar Dijkstra

If we force one weighted score, the result depends on the chosen weights.



Aha

The issue is not Dijkstra's mechanics; it is the single-objective assumption.

4. Pareto Optimality: Intuition

Practical rule

Keep a route unless another route is no worse in every metric and strictly better in at least one.

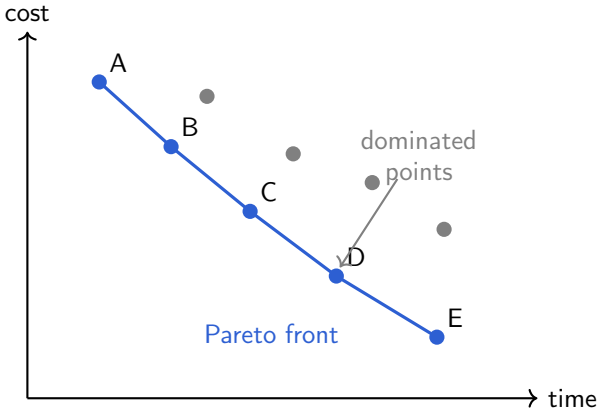
Two valid paths

- Path A: faster, more expensive.
- Path B: slower, cheaper.
- Neither dominates the other.

Why care?

Pareto optimality prevents us from discarding routes that are useful under different preferences.

4. Pareto Optimality: Diagram



Lower time and lower cost are preferred

4. Pareto Optimality: Formal Rule

Dominance

Path p dominates path q when:

$$\forall i,\ C_i(p) \leq C_i(q), \qquad \exists j:\ C_j(p) < C_j(q)$$

Non-dominated set

The Pareto set contains paths that are not dominated by any other feasible path.

Engineering meaning

This rule gives a deterministic filter for keeping useful alternatives.

5. Problem Formulation

Path profile

In this project, each path is represented as:

$$C(p) = (t,\ w,\ tr,\ c)$$

- t: total travel time.
- w: walking distance.
- tr: number of transfers.
- c: monetary cost.

Example route

Walk to metro, ride metro, then use road:

$$(25.3 \text{ min},\ 1.2 \text{ km},\ 2,\ Rs.42.5)$$

Goal

Compute bounded Pareto-optimal paths and return diverse Top-K representatives.

6. System Pipeline

Graph construction → nodes, edges, modes

↓

Cost assignment → weighted multimodal graph

↓

Routing → labels per node

↓

Pareto set → non-dominated destination labels

↓

Top-K selection → diverse representative paths

↓

Visualization → interpretable output

End-to-end view

The data structure evolves from a graph, to labels, to non-dominated paths, to a small visual decision set.

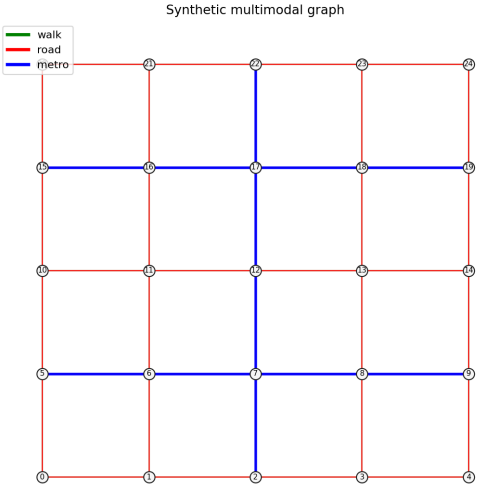
7. Synthetic Network

Network structure

- Grid road network.
- Straight metro lines.
- Walking links between nearby nodes.

Real-world data status

- Travel times available; full network construction in progress
- Data collection and integration across modes is exhaustive
- Focus on delivering one end-to-end result first



8. Cost Modeling

Road

Road cost accounts for distance, fuel, time value, and congestion:

$$c_{\text{road}} = \frac{d}{m} f + \gamma t + \delta \rho$$

d: distance, m: mileage, f: fuel price, ρ: congestion penalty.

Metro and walking

Metro fare:

$$c_{\text{metro}} = \alpha + \beta d$$

Walking:

$$c_{\text{walk}} = 0$$

Why care?

Mode-dependent costs create realistic conflicts between fast, cheap, and low-effort paths.

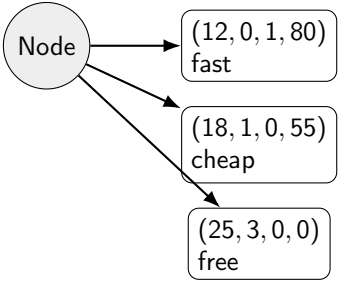
9. Multi-Objective Label Setting: Intuition

Classical shortest path

One best label per node.

Multi-objective version

Keep multiple labels when they represent different trade-offs.



Key intuition

A partial path that is not fastest may still become the cheapest or simplest complete route.

What Happens Internally

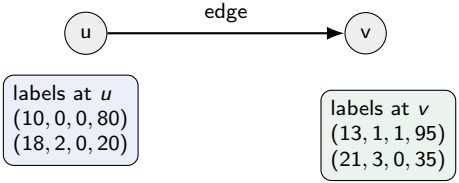
Graph becomes a state space

The algorithm does not only ask "which node am I at?" It asks:

$$\text{node} + \text{current trade-off vector}$$

Each node holds a frontier

A node may store several labels because each label represents a different way to arrive there.



Aha

The algorithm explores the frontier of trade-offs, not just the frontier of nodes.

9. Multi-Objective Label Setting: Mathematical Idea

Label

A label at node v stores a cost vector and predecessor:

$$L = (v,\ C_L,\ \text{prev},\ \text{mode},\ \text{route_id})$$

Propagation over edge $e = (u, v)$

If L is a label at u, generate:

$$C_{L'} = C_L + C_e + (0, 0, \Delta tr, 0)$$

where $\Delta tr = 1$ if the mode or transit route changes.

Engineering rule

Keep L' only if it is feasible and not dominated at node v.

9. Multi-Objective Label Setting: Pseudocode

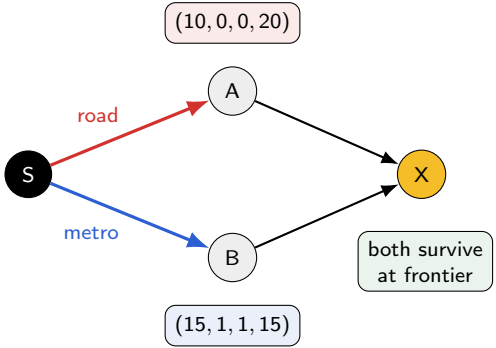
Algorithm 1 Multi-objective Label Setting

```
Initialize labels at source s
Push source label into priority queue
while queue not empty do
    Extract label L
    for each outgoing edge e = (L.node, v) do
        Generate new label L'
        Apply feasibility bounds
        Apply dominance check at node v
        if L' is feasible and non-dominated then
            Insert L'; remove labels dominated by L'
        end if
    end for
end while
Return active labels at destination
```

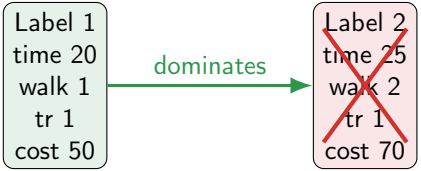
How the Algorithm Builds a Path

Step-by-step expansion

- Start at source: $(0, 0, 0, 0)$
- Expand via road: $(10, 0, 0, 20)$
- Expand via metro: $(15, 1, 1, 15)$
- Compare: both are non-dominated, so keep both.
- Continue expansion from both labels.



10. Dominance: Visual and Math



Why Label 1 dominates

- ✓ better in time: $20 < 25$
- ✓ better in walking: $1 < 2$
- ✓ equal in transfers: $1 = 1$
- ✓ better in cost: $50 < 70$

Formal dominance

$$\forall i,\ C_i(a) \leq C_i(b), \quad \exists j:\ C_j(a) < C_j(b)$$

Why care?

Dominance is the main correctness rule for pruning bad labels without scalarization.

Failure Without Pruning

Label explosion

Without pruning, small trade-offs can create many labels:

$$
\begin{gathered}
(10, 1, 1, 100) \\
(11, 1, 1, 99) \\
(12, 1, 1, 98) \\
(13, 1, 1, 97) \\
\vdots
\end{gathered}
$$

Why this is dangerous

- More labels per node.
- More queue operations.
- More dominance comparisons.
- Slower search and harder interpretation.

Aha

Pareto routing needs both correctness and control.

11. Pruning Strategy

Why needed

Multi-objective routing can create many labels because different partial paths are incomparable.

Feasibility bounds

$$t \leq \alpha_t t^*, \quad w \leq \alpha_w w^*, \quad c \leq \alpha_c c^*$$

Also enforce $tr \leq \alpha_{tr}$.

Interpretation

- t^*, w^*, c^*: single-objective best values.
- α's control tolerance.
- Bounds remove unrealistic detours.

Balance

Tight bounds are fast but may hide alternatives; loose bounds reveal more but cost more.

Complexity Analysis

Classical Dijkstra

One label per node:

$$O((V + E) \log V)$$

Important insight

Worst case can be exponential in the number of objectives because many labels may be mutually non-dominated.

Multi-objective label setting

Let L be the average number of labels per node:

$$O(L \cdot (V + E) \log(LV))$$

Practical case

Dominance checks, feasibility bounds, and Top-K post-selection keep L manageable.

12. Diversity-Based Top-K Selection

Why Top-K?

A Pareto set can contain many valid paths. Users need a compact set of distinct choices.

Selection idea

Start with anchors: min time, min walk, min transfers, min cost. Then add paths that are far from already selected paths.

Max-min rule

$$p^* = \arg\max_{p \in P \setminus S} \min_{q \in S} d(p, q)$$

$$d(p, q) = |\Delta t| + |\Delta w| + |\Delta tr| + |\Delta c|$$

Why care?

We present diverse trade-offs rather than several nearly identical paths.

Algorithm → Code Mapping

Bridge from theory to implementation

Each algorithmic concept is represented by a small, testable code unit.

Concept	Code representation
Label	`class Label`
Cost vector	`(time, walk, transfers, cost)`
Dominance	`dominates(a, b)`
Expansion	edge attribute update
Queue	`heapq` priority queue
Path reconstruction	predecessor pointers
Top-K diversity	greedy max-min selector

Why care?

This mapping keeps the implementation modular: graph building, routing, Pareto utilities, diversity, and visualization remain separate.

13. Implementation: Label Structure

Code idea

Each label stores the node, objective vector, and predecessor pointer for reconstruction.

```
class Label:
    def __init__(self, node, costs, prev=None,
                 previous_mode=None, route_id=None):
        self.node = node
        self.costs = costs      # (time, walk, transfers, cost)
        self.prev = prev
        self.previous_mode = previous_mode
        self.route_id = route_id
        self.active = True
```

Why care?

The predecessor pointer turns final labels back into complete paths.

13. Implementation: Edge Expansion

Core engineering step

Extend a label through an outgoing edge and update transfer count.

```
new_costs = (
    costs[0] + edge["time"],
    costs[1] + edge["walk"],
    costs[2] + transfer_increment,
    costs[3] + edge["cost"],
)
```

Important

The algorithm never collapses objectives into a weighted sum.

14. Results: All Selected Paths

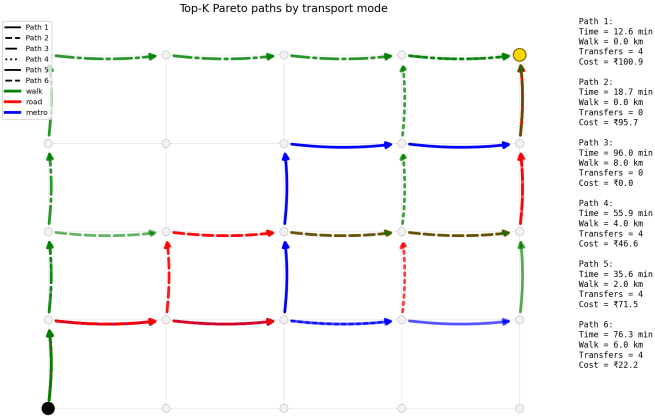
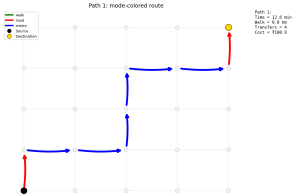


Figure: All selected Pareto paths overlaid on the network.

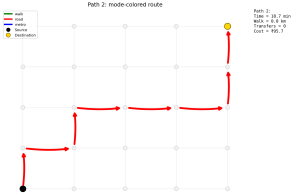
14. Results: Fast and Direct Options

Path 1: The Express



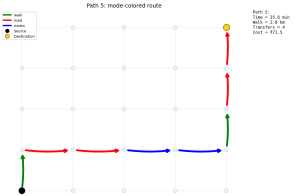
- **Fastest** representative path.
- High cost and transfer-heavy.

Path 2: The Direct



- **Road-only** route.
- No transfers but remains expensive.

Path 5: The Hybrid



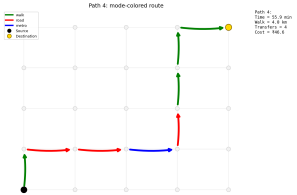
- **Balanced** speed.
- Mid-range cost and effort compromise.

Summary

These routes prioritize time-saving and convenience over monetary cost.

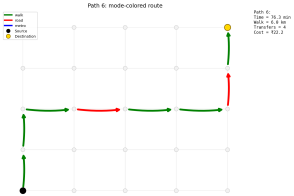
14. Results: Economical and Balanced Options

Path 4: Budget Link



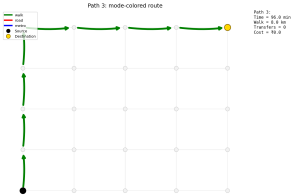
- **Cost-efficient**.
- Moderate walking and travel time.

Path 6: Low-Cost



- **Very Cheap** alternative.
- Long travel time with heavy walking.

Path 3: The Pedestrian



- **Zero Cost**.
- Slowest option; anchors the frontier.

Summary

These routes focus on affordability, ranging from subsidized transit to free walking.

14. Results: Time-Cost Pareto Front

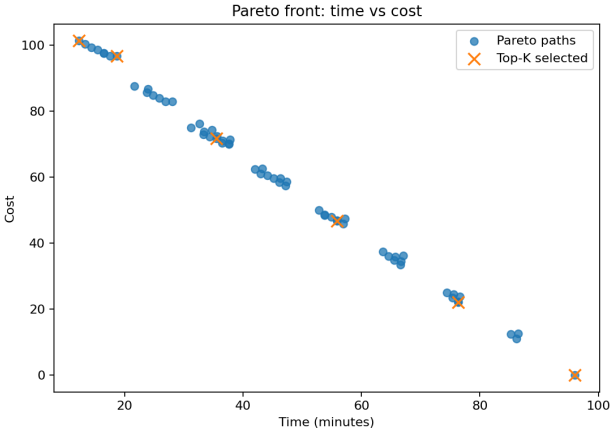


Figure: Computed Pareto frontier in time-cost space.

Insights from Results

Fast paths

- Low time.
- Higher cost.
- Often more mode changes.

Cheap paths

- Low or zero cost.
- More walking.
- Longer travel time.

Balanced paths

- Moderate cost.
- Moderate walking.
- Useful compromise.

visualization explains why the Pareto set has structure.

16. Key Insights

Routing insight

There is often no single best route under multiple objectives.

System insight

Pareto filtering plus Top-K diversity turns a large solution set into actionable choices.

Algorithmic insight

Multi-objective labels preserve trade-offs that scalar shortest path algorithms discard.

User insight

Travelers can compare fast, cheap, low-walk, and low-transfer options explicitly.

17. Conclusion

- Built a synthetic multimodal network with road, metro, and walking modes.
- Modeled time, walking, transfers, and monetary cost.
- Implemented multi-objective label setting with dominance pruning.
- Applied feasibility bounds and diversity-based Top-K selection.
- Produced visual explanations of selected routes and their trade-offs.

Takeaway

Pareto routing gives users a structured set of meaningful choices instead of one opaque answer.

Thank You!

Questions and Discussion

Shreya Ghosh **Soham Chakraborty** **Rahul Dev Sharma**

SL203: Introduction to Algorithms and Software Programming

Instructor: Dr. Nagarajan Natarajan